Quiz Integrador — Projetos de Desenvolvimento de Sistemas

60 questões • 20 fáceis (5 pontos) • 20 médias (10 pontos) • 20 difíceis (15 pontos)
Quatro alternativas por questão, com apenas uma correta. Gabarito ao final.

FÁCIL — QUESTÕES 1 A 20

1. Qual é o objetivo principal do planejamento de um projeto de software?

A) Garantir que nenhuma mudança ocorra.
B) Substituir o levantamento de requisitos.
C) Definir atividades, recursos, prazos e objetivos do projeto.
D) Eliminar a necessidade de testes.

2. O que representa uma entrega em um projeto?

A) Um erro encontrado no código.
B) Uma reunião sem registro.
C) Um resultado verificável produzido pelo trabalho.
D) Uma senha de acesso ao repositório.

3. Qual ferramenta é usada para controlar versões de código?

A) Git.
B) Navegador web.
C) Figma.
D) Postman.

4. Em um quadro Kanban, o que uma coluna geralmente representa?

A) Uma permissão do sistema operacional.
B) Um tipo de banco de dados.
C) Uma etapa do fluxo de trabalho.
D) Uma linguagem de programação.

5. O que é um requisito funcional?

A) A cor obrigatória do computador do desenvolvedor.
B) O nome da pasta de instalação.
C) O orçamento de marketing da empresa.
D) Uma capacidade ou comportamento esperado do sistema.

6. Qual é a função de um cronograma?

A) Executar consultas SQL.
B) Organizar atividades e seus períodos previstos.
C) Desenhar automaticamente interfaces.
D) Criptografar dados pessoais.

7. O que significa trabalhar de forma colaborativa em um projeto?

A) Compartilhar responsabilidades e coordenar contribuições.

B) Evitar revisões entre colegas.

C) Editar sempre o mesmo arquivo sem comunicação.

D) Manter as decisões apenas com uma pessoa.

8. O que é um protótipo de interface?

A) Um serviço de hospedagem.

B) Um relatório de desempenho do servidor.

C) Uma representação preliminar da interação com o sistema.

D) Uma cópia de segurança do banco.

9. Qual é a finalidade de uma reunião de acompanhamento?

A) Excluir tarefas ainda não iniciadas.

B) Verificar progresso, impedimentos e próximos passos.

C) Alterar senhas de todos os usuários.

D) Substituir todos os testes automatizados.

10. O que é um bug em um software?

A) Um modelo de contrato.

B) Uma funcionalidade obrigatória do cliente.

C) Um defeito que provoca comportamento incorreto.

D) Um tipo de licença de hardware.

11. Qual documento pode registrar o escopo de um projeto?

A) Histórico do navegador.

B) Documento de requisitos e delimitação de entregas.

C) Lista de fontes instaladas.

D) Arquivo de cache do sistema.

12. O que caracteriza uma tarefa concluída?

A) Ter sido mencionada em uma conversa.

B) Ter sido atribuída a alguém.

C) Atender aos critérios de conclusão definidos pela equipe.

D) Estar marcada como urgente.

13. Qual é o propósito de um repositório remoto?

A) Dispensar autenticação de usuários.

B) Criar requisitos automaticamente.

C) Compartilhar e armazenar versões do projeto entre colaboradores.

D) Substituir o ambiente de execução.

14. Qual é a finalidade de um teste de software?

A) Escolher o nome do produto.

B) Verificar comportamentos e identificar falhas.

C) Definir salários da equipe.

D) Garantir ausência absoluta de defeitos.

15. O que significa prazo de entrega?

A) Quantidade de linhas de código.

B) Número de usuários simultâneos.

C) Data ou período acordado para concluir uma entrega.

D) Versão do compilador.

16. Qual é a função de um responsável por tarefa?

A) Impedir que colegas consultem a tarefa.

B) Acompanhar e executar ou coordenar sua realização.

C) Aprovar sozinho qualquer mudança de escopo.

D) Eliminar o registro de progresso.

17. O que é uma história de usuário?

A) Descrição breve de uma necessidade sob a perspectiva do usuário.

B) Um registro de acesso ao servidor.

C) Uma sequência de comandos SQL.

D) Uma lista de bibliotecas instaladas.

18. Qual é um exemplo de requisito não funcional?

A) O administrador pode excluir uma conta.

B) O usuário pode cadastrar um produto.

C) O aluno pode responder a um quiz.

D) O sistema deve responder em até dois segundos sob condições definidas.

19. Por que documentar decisões técnicas?

A) Impedir futuras revisões.

B) Facilitar entendimento, manutenção e rastreabilidade.

C) Eliminar a necessidade de comunicação.

D) Substituir a implementação do sistema.

20. O que é uma dependência entre tarefas?

A) Uma variável declarada em código.

B) Uma relação em que uma atividade depende de outra.

C) Uma conexão obrigatória com a internet.

D) Uma cópia duplicada de um arquivo.

MÉDIO — QUESTÕES 21 A 40

21. Qual é a diferença principal entre backlog do produto e backlog da sprint?

A) Ambos são listas de erros exclusivamente.

B) O primeiro registra testes; o segundo substitui requisitos.

C) O primeiro guarda senhas; o segundo guarda códigos-fonte.

D) O primeiro reúne demandas do produto; o segundo seleciona trabalho para a sprint.

22. Quando uma equipe utiliza branches por funcionalidade, qual é o benefício principal?

A) Permitir commits sem histórico.

B) Isolar alterações antes de integrá-las ao código compartilhado.

C) Eliminar conflitos de mesclagem em todos os casos.

D) Evitar qualquer necessidade de revisão.

23. Qual prática reduz o risco de integrar código defeituoso?

A) Apagar o histórico após cada entrega.

B) Desabilitar validações no repositório.

C) Mesclar mudanças diretamente sem verificação.

D) Revisão de pull request acompanhada de testes.

24. O que é um critério de aceitação?

A) Condição verificável para considerar uma funcionalidade atendida.

B) Estimativa informal de salário.

C) Nome escolhido para uma branch.

D) Ferramenta que gera diagramas.

25. Como uma matriz de riscos contribui para o projeto?

A) Substitui a implementação do produto.

B) Determina automaticamente a linguagem ideal.

C) Garante que nenhum risco ocorrerá.

D) Ajuda a avaliar probabilidade, impacto e respostas aos riscos.

26. Em Scrum, qual é a finalidade da retrospectiva?

A) Publicar a versão final sem testes.

B) Definir exclusivamente a arquitetura do banco.

C) Apresentar somente novas funcionalidades ao cliente.

D) Inspecionar a forma de trabalho e propor melhorias.

27. O que caracteriza um MVP?

A) Um software que não exige manutenção.

B) Uma versão mínima capaz de validar hipóteses com usuários.

C) Uma interface sem regras de negócio.

D) Um produto com todas as funcionalidades previstas.

28. Por que usar convenções de commits?

A) Garantir desempenho máximo da aplicação.

B) Impedir que o código seja executado.

C) Eliminar a necessidade de branches.

D) Facilitar a compreensão do histórico e das alterações.

29. Qual é a função de uma pipeline de integração contínua?

A) Automatizar verificações como build e testes após alterações.

B) Garantir que requisitos nunca mudem.

C) Substituir a definição do escopo.

D) Dispensar controle de versões.

30. Quando o cliente solicita uma nova funcionalidade, qual é a primeira medida adequada?

A) Analisar impacto, prioridade e compatibilidade com o escopo.

B) Alterar o cronograma sem comunicar a equipe.

C) Implementar imediatamente sem registro.

D) Excluir uma entrega existente sem consulta.

31. Como diferenciar esforço de duração em uma estimativa?

A) Esforço é sempre igual ao prazo contratual.

B) Esforço mede trabalho necessário; duração mede tempo decorrido.

C) Duração indica exclusivamente custo financeiro.

D) Ambos são sempre medidos em linhas de código.

32. O que é rastreabilidade de requisitos?

A) Capacidade de relacionar requisitos a entregas, código e testes.

B) Processo de remover versões antigas.

C) Forma de distribuir senhas.

D) Método de compressão de imagens.

33. Qual é a utilidade de uma Definition of Done?

A) Definir somente o nome comercial do produto.

B) Determinar o salário dos desenvolvedores.

C) Estabelecer critérios compartilhados para considerar trabalho concluído.

D) Substituir o backlog por mensagens.

34. Qual é uma consequência comum de requisitos ambíguos?

A) Implementações divergentes e retrabalho.

B) Eliminação de defeitos.

C) Redução garantida do custo do projeto.

D) Melhoria automática do desempenho.

35. Em um planejamento incremental, como o produto evolui?

A) Somente por uma entrega única ao final.

B) Por entregas sucessivas que adicionam valor e permitem feedback.

C) Sem validação com interessados.

D) Sem necessidade de priorizar requisitos.

36. O que é débito técnico?

A) Valor de uma licença de software.

B) Custo futuro decorrente de decisões técnicas que exigirão correção ou melhoria.

C) Pagamento devido ao provedor de nuvem.

D) Estimativa de receita do produto.

37. Qual medida ajuda a evitar conflitos recorrentes entre integrantes?

A) Permitir alterações simultâneas sem comunicação.

B) Definir responsabilidades, interfaces e acordos de integração.

C) Desabilitar notificações do projeto.

D) Centralizar todas as tarefas sem registro.

38. Para que serve uma matriz RACI?

A) Esclarecer quem executa, responde, consulta e é informado.

B) Substituir um sistema de versionamento.

C) Classificar tabelas por tamanho.

D) Registrar tempos de resposta de APIs.

39. Como priorizar um defeito crítico que impede o login?

A) Adiar automaticamente para a última entrega.

B) Remover os testes de autenticação.

C) Ignorar o defeito se a interface estiver pronta.

D) Avaliar impacto e urgência e ajustar o backlog conforme o risco.

40. Qual é o benefício de validar protótipos com usuários antes da implementação?

A) Identificar problemas de usabilidade e expectativas antecipadamente.

B) Evitar qualquer mudança futura de requisitos.

C) Eliminar toda necessidade de documentação.

D) Garantir que o código nunca terá falhas.

DIFÍCIL — QUESTÕES 41 A 60

41. Uma equipe concluiu 18 de 24 pontos planejados na sprint. Qual foi a proporção concluída?

A) 60%.

B) 80%.

C) 75%.

D) 90%.

42. Um projeto possui caminho A→B→D com durações 2, 5 e 3 dias, e caminho A→C→D com 2, 4 e 3 dias. Qual é o caminho crítico?

A) A→B→D, com 10 dias.

B) A→C→D, com 9 dias.

C) A→B→D, com 7 dias.

D) Ambos, com 12 dias.

43. Se uma mudança afeta a API consumida pelo front-end, qual estratégia reduz incompatibilidades?

A) Alterar o endpoint sem avisar a equipe.

B) Definir contrato de API, versionar mudanças e testar a integração.

C) Excluir testes de contrato existentes.

D) Publicar a alteração diretamente em produção.

44. Uma funcionalidade passou nos testes unitários, mas falhou na comunicação entre serviços. Qual teste é mais indicado?

A) Teste de integração.

B) Teste de instalação de fontes.

C) Teste de tipografia.

D) Teste de contraste de cores.

45. Ao estimar tarefas com alta incerteza, qual abordagem é mais adequada?

A) Fixar uma data sem avaliar dependências.

B) Ignorar riscos porque a equipe é experiente.

C) Explicitar premissas, decompor trabalho e revisar estimativas com evidências.

D) Somar sempre um dia a qualquer tarefa.

46. Uma equipe percebe que tarefas chegam a teste apenas no último dia da sprint. Qual melhoria tende a reduzir esse gargalo?

A) Aumentar indefinidamente as tarefas iniciadas.

B) Suspender revisões até a entrega final.

C) Limitar trabalho em andamento e integrar/testar continuamente.

D) Remover os critérios de aceitação.

47. Qual é a principal vantagem de relacionar cada requisito a casos de teste?

A) Garantir ausência de defeitos em produção.

B) Identificar requisitos sem cobertura e avaliar impactos de mudanças.

C) Substituir os testes por documentação.

D) Eliminar a necessidade de ambientes de teste.

48. Se o custo de corrigir uma falha cresce após a implantação, qual prática ajuda a detectá-la mais cedo?

A) Eliminar validação de requisitos.

B) Executar testes somente após a entrega ao cliente.

C) Revisões e testes desde as etapas iniciais do desenvolvimento.

D) Evitar feedback durante a implementação.

49. Uma equipe adota trunk-based development. Qual prática é coerente com esse modelo?

A) Eliminar a execução de testes no merge.

B) Manter branches isoladas por muitos meses sem integração.

C) Integrar alterações pequenas e frequentes à linha principal com verificações automatizadas.

D) Criar uma branch principal diferente por desenvolvedor.

50. Qual é a função de um plano de rollback em uma implantação?

A) Definir como retornar a um estado estável se a publicação falhar.

B) Substituir o backup dos dados.

C) Garantir que nunca haverá incidentes.

D) Eliminar a necessidade de monitoramento.

51. Em uma aplicação com dados pessoais, qual decisão deve ocorrer já no planejamento?

A) Armazenar senhas em texto simples.

B) Publicar registros reais no repositório.

C) Coletar todos os dados disponíveis por precaução.

D) Definir finalidade, minimização, controle de acesso e proteção dos dados.

52. Uma dependência externa apresenta falhas frequentes. Qual ação é mais apropriada?

A) Assumir disponibilidade permanente.

B) Ignorar o impacto nos requisitos.

C) Remover o problema do relatório de riscos.

D) Registrar o risco e planejar tratamento, como contingência e monitoramento.

53. Quando duas equipes implementam a mesma funcionalidade sem saber, qual falha de gestão está mais evidente?

A) Ausência de visibilidade e coordenação do trabalho.

B) Excesso de critérios de aceitação.

C) Uso obrigatório de banco relacional.

D) Existência de testes automatizados.

54. Como avaliar se uma melhoria de desempenho atingiu seu objetivo?

A) Observar somente a aparência da interface.

B) Comparar métricas medidas antes e depois sob condições equivalentes.

C) Verificar quantos commits foram realizados.

D) Contar apenas linhas de código alteradas.

55. Qual situação caracteriza mudança de escopo não controlada?

A) Registrar uma decisão em ata.

B) Atualizar uma dependência conforme procedimento aprovado.

C) Corrigir um defeito previsto no plano.

D) Adicionar funcionalidades sem análise de impacto ou aprovação pertinente.

56. Uma funcionalidade depende de autenticação e de uma API ainda não implementadas. O que deve orientar o planejamento?

A) Ignorar a API até o teste de aceitação.

B) Mapear dependências e sequenciar entregas ou definir contratos e simulações.

C) Iniciar a publicação final imediatamente.

D) Eliminar a autenticação para cumprir o prazo.

57. Um requisito exige disponibilidade de 99,9% em 30 dias de 24 horas. Aproximadamente quanto tempo de indisponibilidade esse limite permite?

A) 7,2 horas.

B) 43,2 minutos.

C) 72 minutos.

D) 4,32 minutos.

58. Em uma revisão de segurança, identificou-se uma biblioteca vulnerável usada em produção. Qual resposta é adequada?

A) Remover os testes de segurança da pipeline.

B) Avaliar exposição e criticidade, corrigir ou mitigar e validar a atualização.

C) Aguardar indefinidamente sem registrar o risco.

D) Desabilitar logs para ocultar alertas.

59. Uma entrega tem critérios de aceitação satisfeitos, mas falha em uma verificação obrigatória de segurança. Ela está pronta?

A) Sim; falhas de segurança são sempre irrelevantes.

B) Sim; critérios funcionais dispensam segurança.

C) Não; o critério de conclusão deve incluir a verificação de segurança exigida.

D) Sim; basta a aprovação de um desenvolvedor.

60. Qual prática permite auditar a origem de uma alteração entregue?

A) Apagar branches e commits antes da implantação.

B) Renomear arquivos sem registrar mudanças.

C) Compartilhar o código apenas por mensagens privadas.

D) Relacionar issue, pull request, revisão, commit e versão publicada.

Gabarito

Fácil

01: C | 02: C | 03: A | 04: C | 05: D
06: B | 07: A | 08: C | 09: B | 10: C
11: B | 12: C | 13: C | 14: B | 15: C
16: B | 17: A | 18: D | 19: B | 20: B

Médio

21: D | 22: B | 23: D | 24: A | 25: D
26: D | 27: B | 28: D | 29: A | 30: A
31: B | 32: A | 33: C | 34: A | 35: B
36: B | 37: B | 38: A | 39: D | 40: A

Difícil

41: C | 42: A | 43: B | 44: A | 45: C
46: C | 47: B | 48: C | 49: C | 50: A
51: D | 52: D | 53: A | 54: B | 55: D
56: B | 57: B | 58: B | 59: C | 60: D